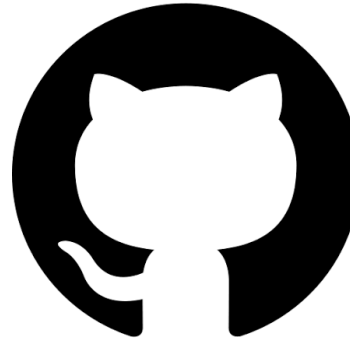




GitHub

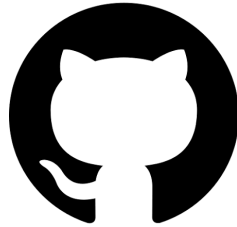


GitHub is a collaboration platform built on top of a distributed version control system.





Why ?

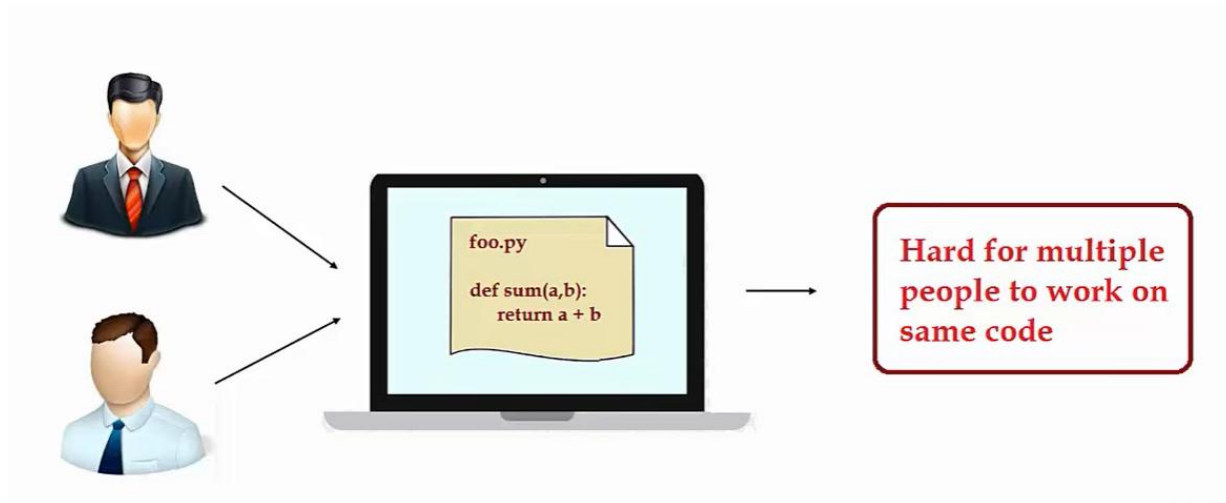


GitHub

Problem # 1



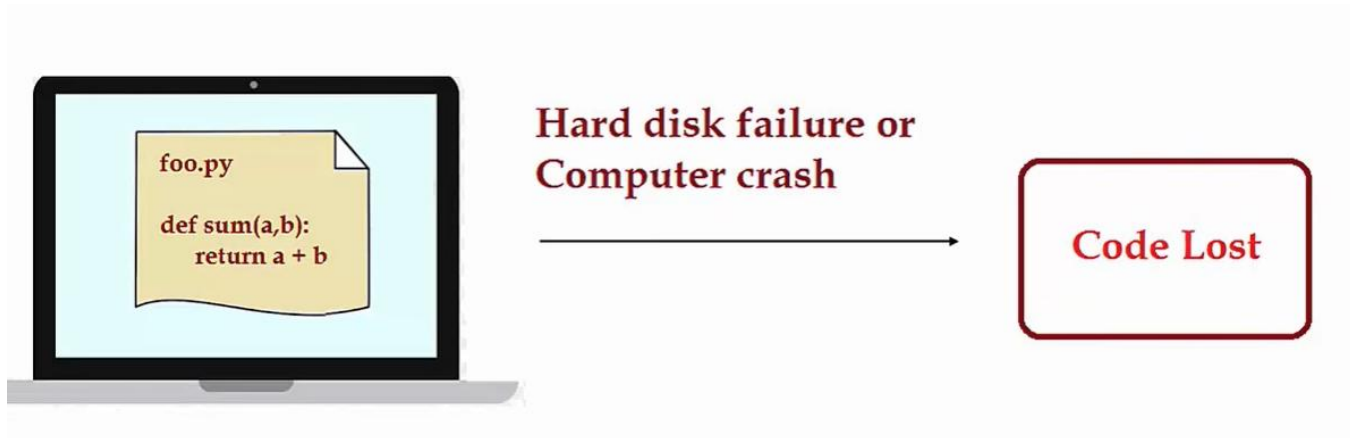
Problem # 2



Problem #3



Problem #1



Solution #1



Git saves your code changes over time, so you never lose your work.

THE PROBLEM

Hard disk failure or
Computer crash

Code Lost

THE SOLUTION: GIT

Push (Save)

Pull (Get Back)

GitHub
(Remote Repository)

Your Computer
(Local Repository)

Your code and
all changes are
safely stored
in the cloud.

THE RESULT

Even if your computer crashes,
your code is safe!

Code Safe & Restored

HOW IT WORKS (EASY STEPS)

1. You Write Code

You work on your files.

2. Save Changes

Git takes a snapshot
of your changes.

3. Push to GitHub

Your changes are
uploaded to GitHub.

4. Safe in the Cloud

Stored safely and can be
accessed anytime.

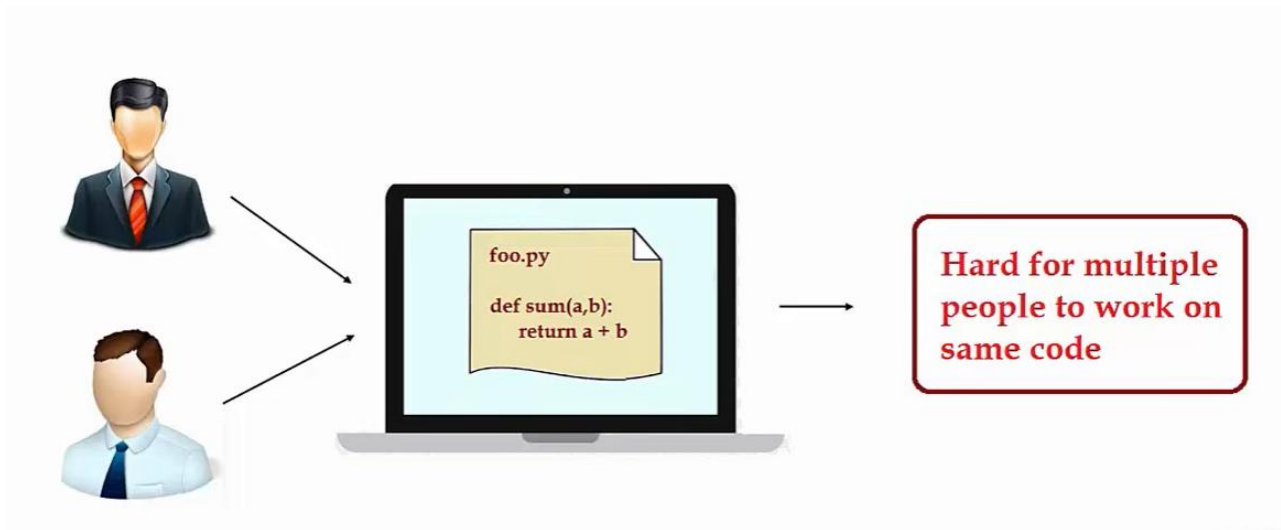
5. Get Back Anytime

Pull your code anytime and
continue your work.



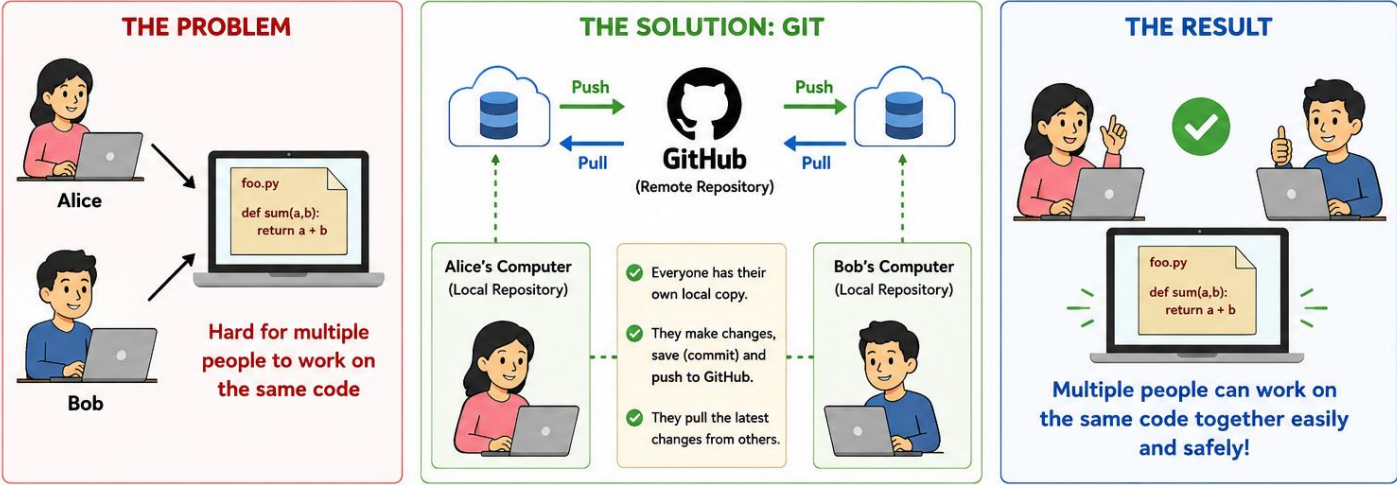
Key Idea: Git keeps a history of all your changes in the cloud, so you can always go back and restore your code.

Problem #2

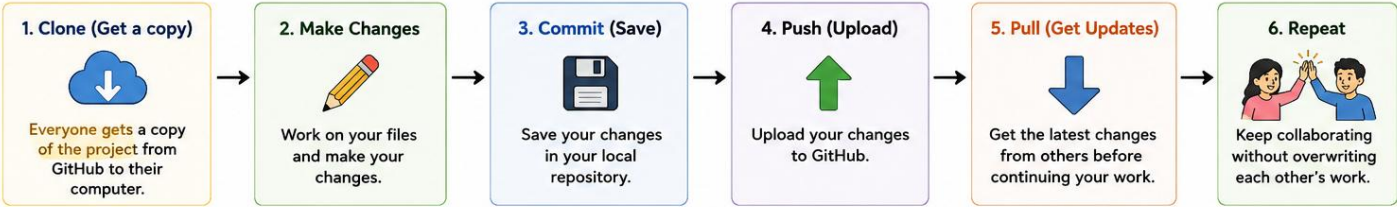


Solution # 2

✔ Git allows multiple people to work on the same code safely and without overwriting each other's work.



HOW IT WORKS (EASY STEPS)



💡 **Key Idea:** Git keeps track of all changes and helps everyone work together on the same code smoothly.

Problem # 3

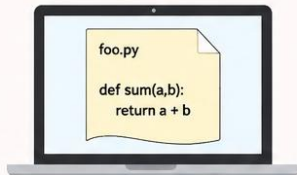


Solution # 3

✓ Git keeps a history of all changes, so you can easily track changes and go back to any previous version.

THE PROBLEM

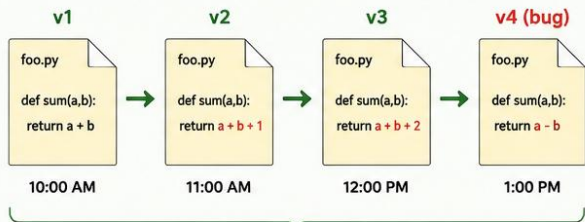
Hard to track changes
Revert back to previous version



What did I change?
How do I go back?

THE SOLUTION: GIT

Git keeps every change as a version (snapshot).



🕒 Git History (All changes are saved)

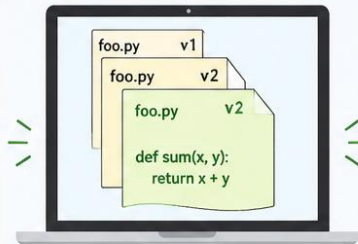
Want to go back?

You can easily revert to any
previous version with one command.



THE RESULT

You can see all versions and
go back to any working version.



Track changes easily
Revert to previous
versions anytime!

HOW IT WORKS (EASY STEPS)

1. Make Changes



You edit your code.

2. Commit (Save)



Git saves a snapshot
of your changes.

3. History is Created



Each commit becomes
a version in history.

4. View History



You can see all
previous versions.

5. Revert (Go Back)



Choose a previous
version and revert.

6. Back to Working



You're back to a working
version!

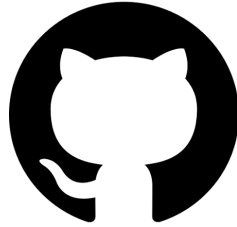


Key Idea: Git saves every change. You can compare, track, and go back anytime without losing your work.



Core reasons to use **GitHub**

- Version control & history
- Collaboration without conflicts
- Backup & central hub
- Code review & quality
- Automation (CI/CD)
- Ecosystem & integrations

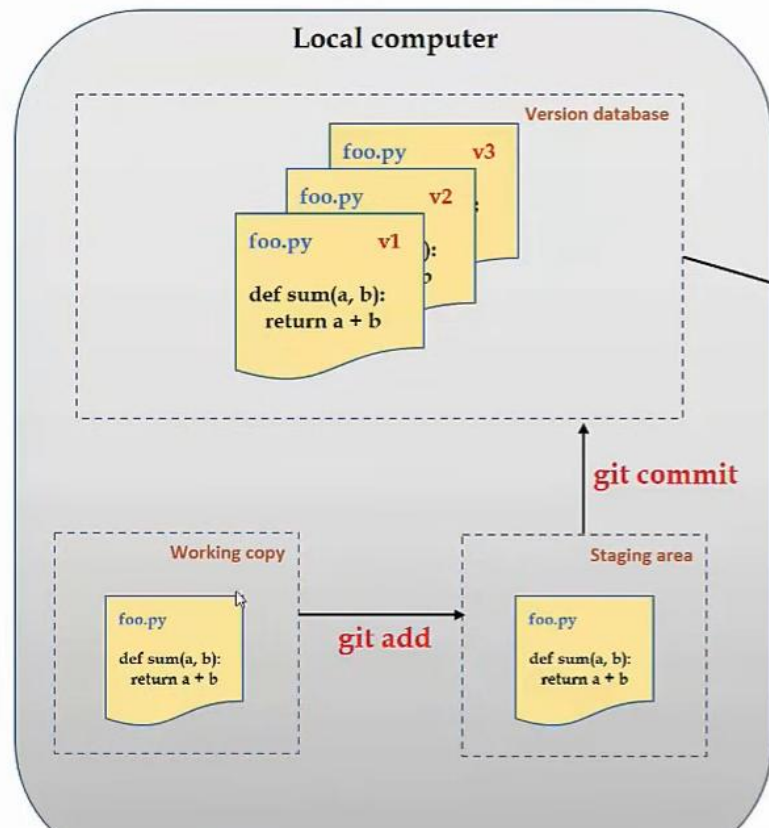


What is
GitHub

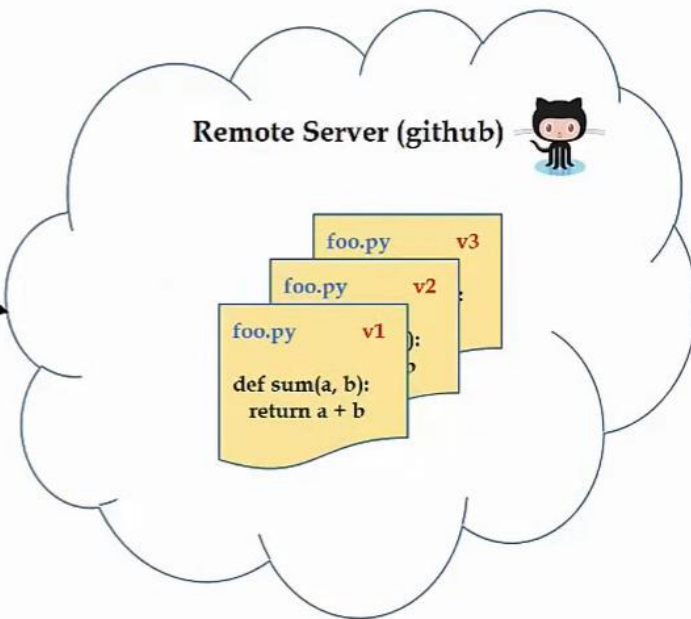


GitHub is

a website that allows you to store your code on the cloud. It is not just a cloud storage but a full version control system powered by git.



git push





Commands:

clone

status

add

commit

push

log

restore

revert / reset



GIT COMMANDS

CHEAT SHEET FOR BEGINNERS



What is Git?

Git is a distributed version control system that helps you track changes in your code and collaborate efficiently.



1. SETUP & CONFIGURATION

Command	Description
git --version	Check Git version
git config --global user.name "Your Name"	Set your username
git config --global user.email "you@example.com"	Set your email
git config --list	List all configuration



4. COMMIT CHANGES

Command	Description
git commit -m "message"	Commit staged changes with message
git commit -am "message"	Add & commit changes with message



5. BRANCHING

Command	Description
git branch	List all branches
git branch <branch-name>	Create a new branch
git checkout <branch-name>	Switch to a branch
git checkout -b <branch-name>	Create and switch to a new branch
git branch -d <branch-name>	Delete a merged branch
git branch -D <branch-name>	Force delete a branch



2. CREATE & CLONE REPOSITORY

Command	Description
git init	Initialize a new Git repository
git clone <repo-url>	Clone (copy) a repository from remote



3. WORKING WITH FILES

Command	Description
git status	Check status of files
git add <file>	Stage a specific file
git add .	Stage all changes
git restore <file>	Discard changes in a file
git rm <file>	Remove a file from working directory
git rm --cached <file>	Unstage a file



6. MERGING

Command	Description
git merge <branch-name>	Merge a branch into current branch



7. REMOTE REPOSITORY

Command	Description
git remote -v	Show remote repositories
git remote add origin <repo-url>	Add remote repository
git remote remove origin	Remove remote repository
git push origin <branch-name>	Push changes to remote
git push -u origin <branch-name>	Push & set upstream branch
git pull origin <branch-name>	Pull changes from remote
git fetch	Fetch latest changes (from remote)



9. UNDOING CHANGES

Command	Description
git restore <file>	Discard changes in file
git restore --staged <file>	Unstage a file
git reset HEAD <file>	Unstage a file (older way)
git reset --soft <commit-id>	Undo commit (keep changes)
git reset --hard <commit-id>	Undo commit (remove changes)
git revert <commit-id>	Create new commit to undo changes



8. VIEWING HISTORY

Command	Description
git log	Show commit history
git log --oneline	Show short commit history
git log --graph --oneline --all	Show branch history graph
git show <commit-id>	Show details of a commit
git diff	Show changes not staged
git diff --staged	Show changes staged for commit

10. IMPORTANT POINTERS (CONCEPTS)



HEAD

Points to the current commit you are working on.



Working Directory

Files in your local folder that are not yet staged.



Staging Area (Index)

Area where staged changes are kept before commit.



Repository (.git folder)

Stores all history and configuration of the project.



Remote Repository

Version of your project hosted on platforms like GitHub, GitLab.

BASIC GIT WORKFLOW



Working Directory



git add



Staging Area (Index)



git commit



Remote Repository (git push)

TIPS FOR BEGINNERS

- Always commit small and meaningful changes.
- Pull before you push to avoid conflicts.
- Use meaningful commit messages.
- Check status frequently with `git status`.
- Practice regularly on your own projects.



What is **git clone**?

Creates a **local copy** of a remote Git repository on your computer.

What it does:

-  Downloads the entire repository
-  Creates a local project folder
-  Includes all branches and commit history
-  Connects the local repository to the remote (**origin**)

Ex: `git clone https://github.com/username/repository.git`

Why use it?

Use **git clone** when you want to start working on an existing GitHub project or collaborate with a team.



What is `git status`?

Displays the current state of your Git repository.

Command : `git status`

It shows:

-  Modified files
-  New (untracked) files
-  Staged files (ready to commit)
-  Branch information

Why use it?

Always run `git status` before committing to know exactly what will be included in your next commit.



What is `git add`?

Moves changes from the **Working Directory** to the **Staging Area**, preparing them for the next commit.

Command : `git add <filename>`

It shows:

- + Stages new files
-  Stages modified files
-  Prepares changes for a commit
-  Lets you choose exactly what to commit

Ex: `git add app.py` # Stage a specific file
`git add .` # Stage all changes

Why use it?

Use `git add` to select the changes you want to include in your next commit before running `git commit`.



What is **git commit**?

Saves the **staged changes** as a permanent snapshot in your local Git repository.

Command : `git commit -m "Add login feature"`

It shows:

-  Saves staged changes permanently
-  Adds a descriptive commit message
-  Creates a new version in the project history
-  Makes it easy to track and revert changes

Ex: `git add app.py`
`git commit -m "Fix login validation bug".`

Why use it?

Commit frequently with clear messages to keep your project history organized and easy to understand.



What is **git push**?

Uploads your **local commits** to a **remote repository** (e.g., GitHub)

Command : **git push origin main**

What it does:

-  Uploads local commits to the remote repository
-  Shares your changes with your team
-  Updates the remote branch
-  Enables collaboration and backups

Ex: **git** add app.py
git commit -m "Fix login validation bug".
git push origin main

Why use it?

Use **git push** after committing your changes to publish your work to GitHub or another remote repository.



What is **git push**?

Uploads your **local commits** to a **remote repository** (e.g., GitHub)

Command : **git push origin main**

What it does:

- ↑ Uploads local commits to the remote repository
- 🌐 Shares your changes with your team
- ↻ Updates the remote branch
- 👉 Enables collaboration and backups

Ex: **git** add app.py
git commit -m "Fix login validation bug".
git push origin main

Why use it?

Use **git push** after committing your changes to publish your work to GitHub or another remote repository.



What is **git log**?

Displays the **commit history** of your Git repository.

Command : **git log**

What it shows:

-  Commit ID (SHA)
-  Author name
-  Date & time of the commit
-  Commit message
-  Complete project history

Ex: **git log**

Why use it?

Use **git log** to review project history, track changes, identify commits, and debug issues effectively.



What is **git restore**?

Restores a file to its previous state by discarding unwanted changes in the **Working Directory**.

Command : `git restore <file_name>`

What it does:

-  Discards uncommitted changes
-  Restores the file to the last committed version
-  Works on specific files
-  Does not affect committed history

Ex: `git restore app.py`

Why use it?

Use **git log** to review project history, track changes, identify commits, and debug issues effectively.



What is **git restore**?

Restores a file to its previous state by discarding unwanted changes in the **Working Directory**.

Command : `git restore <file_name>`

What it does:

-  Discards uncommitted changes
-  Restores the file to the last committed version
-  Works on specific files
-  Does not affect committed history

Ex: `git restore app.py`

Why use it?

Use **git restore** when you want to undo changes in a file before committing them. Be careful—discarded changes cannot be recovered unless they were previously committed.



What is **git revert**?

Creates a **new commit** that undoes the changes introduced by a previous commit, without deleting the project history.

Command : `git revert <commit_id>`

What it does:

-  Reverses the changes of a specific commit
-  Creates a new commit with the undo changes
-  Preserves the complete commit history
-  Safe for shared repositories

Ex: `git revert a1b2c3d`

Why use it?

Use **git revert** to safely undo a committed change while keeping the project's history intact, especially when collaborating with others.



What is **git branch**?

Creates, lists, or deletes **branches**, allowing you to work on features independently without affecting the main codebase.

Command : **git branch**

What it does:

-  Creates new branches
-  Lists available branches
-  Deletes completed branches
-  Enables parallel development

Ex: **git branch feature-login** # Create a new branch
git branch # List all local branches
git branch -d feature-login # Delete a branch

Why use it?

Use **git branch** to organize development, work on new features safely, and collaborate without impacting the main branch.



What is **git checkout**?

Switches between **branches** or restores files from another branch or commit.

Command : `git checkout <branch_name>`

What it does:



Switches to another branch



Creates and switches to a new branch (**-b**)



Lets you work on different versions of your project



Enables feature development in isolation

Ex: `git checkout feature-login` # Switch to an existing branch
`git checkout -b feature-login` # Create and switch to a new branch

Why use it?

Use **git checkout** to move between branches and work on different features without affecting the main codebase.



What is **git merge**?

Combines the changes from one **branch** into another, integrating completed work into your project.

Command : **git merge** <branch_name>

What it does:

-  Combines two branches
-  Preserves commit history
-  Integrates completed features
-  May require resolving merge conflicts

Ex: **git** checkout main
git merge feature-login

Why use it?

Use **git merge** to combine feature branches into the main branch after development and testing are complete.



What is **git pull**?

Fetches the latest changes from a **remote repository** and merges them into your current local branch

Command : **git pull origin main**

What it does:

- ↓ Downloads the latest changes
- ↻ Merges remote updates into your local branch
- 👉 Keeps your project up to date
- ⚠ May require resolving merge conflicts

Ex: **git pull origin main**

Why use it?

Use **git pull** before starting work or before pushing your changes to ensure your local repository is synchronized with the remote repository..



What is **git delete**?

Deletes a **local branch** that has already been merged.

Command : `git branch -d <branch_name>`

What it does:



Deletes a local branch



`-d` deletes only merged branches



`-D` force deletes unmerged branches



Keeps your repository clean

Ex: `git branch -d feature-login` # Delete a merged branch
`git branch -D feature-login` # Force delete an unmerged branch

Why use it?

Delete feature branches after they have been merged to keep your repository organized.



Q

&

A